Staged Basic Program Loader

Foundation

Original code

```
10 REM by Jan De Beer, 2023
20 CLEAR: SYMBOL AFTER 32
30 IF A > 0 GOTO 200
40 PRINT "Hello"
50 GOTO 50
60 GOSUB 210
```

Staged code

(4)	(3) (9)	10 ON ERROR GOTO 50:ON BREAK GOSUB 50:REM @interrupt
	(3) (10)	20 SYMBOL AFTER 256:MEMORY &a66b:REM @himem
	(3) (8)	30 GOTO 80:REM @jump
	(3) (7)	40 ERROR 255:REM @error
	(3) (6)	50 POKE &a67b,1:POKE &eeee,ERR:END:REM @end
	(3)	60 REM @preamble
	(3)	70 REM @preamble
	(3) (5)	80 REM @land
		90 REM by Jan De Beer, 2023
		100 CLEAR:ON ERROR GOTO 50:ON BREAK GOSUB 50: SYMBOL AFTER 32
		110 IF A > 0 GOTO 160
		120 PRINT "Hello"
		130 GOTO 130
		140 GOSUB 170
	(1)	150 GOTO 150:REM @bridge

	(2)	`160 GOTO 0:REM @link`
	(2)	`170 GOTO 0:REM @link`

Macros

(1) **ProgramBridgeMacro** (LOCAL) ⇐ ProgramBridgeBasicPreprocessor
(2) **DynamicLinkMacro** (LOCAL) ⇐ ProgramBridgeBasicPreprocessor
(3) **PreambleLineMacro** (GLOBAL) ⇐ PreambleBasicPreprocessor
(4) **PreambleBlockMacro** (GLOBAL) ⇐ PreambleBasicPreprocessor
(5) **PreambleLandingMacro** (GLOBAL) ⇐ PreambleLandingBasicPreprocessor
(6) **EndingMacro** (GLOBAL) ⇐ EndingBasicPreprocessor
(7) **ErrorOutMacro** (GLOBAL) ⇐ ErrorOutBasicPreprocessor
(8) **PreambleJumpingMacro** (GLOBAL) ⇐ PreambleJumpingBasicPreprocessor
(9) **InterruptMacro** (GLOBAL) ⇐ InterruptBasicPreprocessor
(10) **HimemMacro** (GLOBAL) ⇐ HimemBasicPreprocessor

Macro handlers

EndingMacroHandler (GLOBAL)

Memory trap on &a67b (GLOBAL)
Handling:

1. Wait until Ready
2. PEEK &eeee to get error code (0 if end without error)
3. Dispose program runtime (trap gets removed), passing error code
4. Code disclosure
5. Ending action, if any (for example, return to program browser)

ⓘ The END instruction is inside a macro (not inline), making sure it is left untouched by *EndingBasicPreprocessor*

LOAD/SAVE Binary

Original code

```
90 REM by Jan De Beer, 2023
100 CLS: LOAD "filename",startAddress
```

likewise

```
100 CLS: SAVE "filename",B,startAddress,lengthInBytes
```

Staged code

(4)	(3) (9)	10 ON ERROR GOTO 50:ON BREAK GOSUB 50:REM @interrupt
	(3) (10)	20 SYMBOL AFTER 256:MEMORY &a66b:REM @himem
	(3) (8)	30 GOTO 80:REM @jump
	(3) (7)	40 ERROR 255:REM @error
	(3) (6)	50 POKE &a67b,1:POKE &eeee,ERR:END:REM @end
	(3) (11)	60 IF PEEK(&aaaa)=0 GOTO 60:REM @waitresume
	(3) (11)	70 POKE &aaaa,0:RETURN:REM @waitresume
	(3) (5)	80 REM @land
		90 REM by Jan De Beer, 2023
		100 CLS: POKE &bbbb,1:GOSUB 60
		...

Macros

(11) **WaitResumeMacro** (GLOBAL) ⇐ WaitResumeBasicPreprocessor

Macro handlers

BinaryLoadMacroHandler (LOCAL)

Contains a map of source instructions by reference
Memory trap on &bbbb (LOCAL)
Handling:

1. Lookup source instruction by reference (trap value &bbbb)
2. Lookup destination file
3. Perform memory load
4. Resume by POKE &aaaa,1 (GLOBAL from *WaitResumeMacro*)

BinarySaveMacroHandler (LOCAL)

Contains a map of source instructions by reference
Memory trap on &bbbb (LOCAL)
Handling:

1. Lookup source instruction by reference (trap value &bbbb)
2. Lookup destination file
3. Perform memory save
4. Resume by POKE &aaaa,1 (GLOBAL from *WaitResumeMacro*)

OPENOUT

Original code

```
90 REM by Jan De Beer, 2023
100 OPENOUT "file"
110 PRINT #9, "Hallo"
120 A$="world": PRINT #9, A$
130 CLOSEOUT
```

Staged code

(4)	(3) (9)	`10 ON ERROR GOTO 50:ON BREAK GOSUB 50:REM @interrupt`
	(3) (10)	`20 SYMBOL AFTER 256:MEMORY &a66b:REM @himem`
	(3) (8)	`30 GOTO 80:REM @jump`
	(3) (7)	`40 ERROR 255:REM @error`
	(3) (6)	`50 POKE &a67b,1:POKE &eeee,ERR:END:REM @end`
	(3) (11)	`60 IF PEEK(&aaaa)=0 GOTO 60:REM @waitresume`
	(3) (11)	`70 POKE &aaaa,0:RETURN:REM @waitresume`
	(3) (5)	`80 REM @land`
		`90 REM by Jan De Beer, 2023`
		`100 POKE &bbbb,1:GOSUB 60`
		`110 POKE &bbbb,2:GOSUB 60`

		120 A$="world": POKE &bbbb,3:GOSUB 60
		130 POKE &bbbb,4:GOSUB 60
		...

Macro handlers

TextSaveMacroHandler (LOCAL)

Contains a map of source instructions by reference
Memory trap on &bbbb (LOCAL)
Handling:

1. Lookup source instruction by reference (trap value &bbbb)
2. IF “OPENOUT”
    a. Open file writer
3. ELSE IF “PRINT”
    a. Println literal or variable to file writer
4. ELSE IF “CLOSEOUT”
    a. Flush and close file writer
5. Resume by POKE &aaaa,1 (GLOBAL from *WaitResumeMacro*)

OPENIN

Original code

```
90 REM by Jan De Beer, 2023
100 OPENIN “file”
110 LINE INPUT #9, H$
120 WHILE EOF=0
130 INPUT #9, D$
140 PRINT D$
150 WEND
160 CLOSEIN
```

Staged code

(4)	(3) (9)	10 ON ERROR GOTO 50:ON BREAK GOSUB 50:REM @interrupt
	(3) (10)	20 SYMBOL AFTER 256:MEMORY &a66b:REM @himem

	(3) (8)	30 GOTO 80:REM @jump
	(3) (7)	40 ERROR 255:REM @error
	(3) (6)	50 POKE &a67b,1:POKE &eeee,ERR:END:REM @end
	(3) (11)	60 IF PEEK(&aaaa)=0 GOTO 60:REM @waitresume
	(3) (11)	70 POKE &aaaa,0:RETURN:REM @waitresume
	(3) (5)	80 REM @land
		90 REM by Jan De Beer, 2023
		100 DAT$=SPACE$(255):DAT%=0:EOD%=0:POKE &bbbb,1:GOSUB 60
		110 POKE &bbbb,2:GOSUB 60:H$=LEFT$(DAT$,DAT%)
		120 WHILE EOD%=0
		130 POKE &bbbb,2:GOSUB 60:D$=LEFT$(DAT$,DAT%)
		140 PRINT D$
		150 WEND
		160 POKE &bbbb,3:GOSUB 60:DAT$=""
		...

Macro handlers

TextLoadMacroHandler (LOCAL)

Contains a map of source instructions by reference
Memory trap on &bbbb (LOCAL)
Handling:

1. Lookup source instruction by reference (trap value &bbbb)
2. IF “OPENIN”
    a. Open file reader
    b. Pre-read first line from line reader and initialize EOD% variable
3. ELSE IF “INPUT” OR “LINE INPUT”
    a. Fill read line in the string variable DAT$
    b. Fill line length in the integer variable DAT%
    c. Pre-read next line from file reader and update EOD% variable
4. ELSE IF “CLOSEIN”

a. Close file reader

5. Resume by POKE &aaaa,1 (GLOBAL from *WaitResumeMacro*)

CHAIN RUN

Original code

```
90 REM by Jan De Beer, 2023
100 CLS: RUN "
```

likewise

```
100 CLS: RUN "program"
100 CLS: CHAIN "program"[,lineNumber]
```

Staged code

(4)	(3) (9)	10 ON ERROR GOTO 50:ON BREAK GOSUB 50:REM @interrupt
	(3) (10)	20 SYMBOL AFTER 256:MEMORY &a66b:REM @himem
	(3) (8)	30 GOTO 80:REM @jump
	(3) (7)	40 ERROR 255:REM @error
	(3) (6)	50 POKE &a67b,1:POKE &eeee,ERR:END:REM @end
	(3) (12)	60 IF PEEK(&aaaa)=0 GOTO 60:REM @chainrun
	(3) (12	70 POKE &aaaa,0:RUN 10:REM @chainrun
	(3) (5)	80 REM @land
		90 REM by Jan De Beer, 2023
		100 CLS: POKE &bbbb,1:GOTO 60
		...

Macros

(12) **ChainRunMacro** (GLOBAL) ⇐ ChainRunBasicPreprocessor

Macro handlers

ChainRunMacroHandler (LOCAL)

Contains a map of source instructions by reference
Memory trap on &bbbb (LOCAL)
Handling:

1. Lookup source instruction by reference (trap value &bbbb)
2. Lookup destination file
3. Same loader, load the chained program
    a. Disposes the current program runtime (traps get removed)
4. On the new program runtime, run in special “CHAINRUN” mode
    a. No actual run, but traps get installed
5. Define starting line number, either mapped from instruction or smallest line number
6. Edit run line number in *ChainRunMacro*
7. Swap new code
8. POKE &aaaa,1 (GLOBAL from *ChainRunMacro*)

CHAIN MERGE

Original code

```
90 REM by Jan De Beer, 2023
100 CLS: CHAIN MERGE “program”[,lineNumber][,DELETE lineNumberRange]
```

Staged code

(4)	(3) (9)	10 ON ERROR GOTO 50:ON BREAK GOSUB 50:REM @interrupt
	(3) (10)	20 SYMBOL AFTER 256:MEMORY &a66b:REM @himem
	(3) (8)	30 GOTO 80:REM @jump
	(3) (7)	40 ERROR 255:REM @error
	(3) (6)	50 POKE &a67b,1:POKE &eeee,ERR:END:REM @end
	(3) (13)	60 IF PEEK(&aaaa)=0 GOTO 60:REM @chainmerge
	(3) (13)	70 POKE &aaaa,0:RESTORE:GOTO 0:REM @chainmerge
	(3) (5)	80 REM @land

		`90 REM by Jan De Beer, 2023`
		`100 CLS: POKE &bbbb,1:GOTO 60`
		`...`

Macros

(13) **ChainMergeMacro** (GLOBAL) ⇐ ChainMergeBasicPreprocessor

ⓘ The *ChainMergeMacro* is upfront (before the first original code line), making sure it remains at a fixed position in the byte code no matter what manipulations happen with the (chained) code

Macro handlers

ChainMergeMacroHandler (LOCAL)

Contains a map of source instructions by reference
Memory trap on &bbbb (LOCAL)

Define <G> as the global, original to staged line number mapping
The domain of <G> is the original (merged) source code, meaning excluding any macros

Handling:

1. Lookup source instruction by reference (trap value &bbbb)
2. Lookup destination file
3. Preprocess chained code
    a. Create new empty session
    b. Apply *ProgramBridgeBasicPreprocessor* with new session
    c. Apply *InterruptBasicPreprocessor* with new session
    d. Renum to not overlap current code => line mapping <M>
    e. Renum macros in new session using <M>
    f. Amend <G> = <G> union (<M> minus macros)
4. Bridge programs in current code
    a. Edit *ProgramBridgeMacro* to GOTO first renum line number of chained code
    b. Remove *ProgramBridgeMacro*
5. Merge
    a. Remove any DELETE lines from the (renumbered) current code (no impact on macros)
    b. Merge code
    c. Merge macros (copy macros from new session to current session)
    d. Extend list of chained programs in current session

6. Preprocess the entire merged code
    a. Apply *LinkResolveBasicPreprocessor* on every *DynamicLinkMacro* whose original line number is inside the domain of <G>
        i. Edit *DynamicLinkMacro* to GOTO original line number mapped by <G>
        ii. Remove *DynamicLinkMacro*
    b. Apply *BinaryLoadBasicPreprocessor*
    c. Apply *BinarySaveBasicPreprocessor*
    d. Apply *ChainRunBasicPreprocessor*
    e. Apply *ChainMergeBasicPreprocessor*
    f. Apply *EndingBasicPreprocessor*
7. Edit resume line number in *ChainMergeMacro*
    a. When source instruction defines a line number, use it (via dynamic link)
    b. Otherwise set to the first line number of the merged code
8. Swap merged code
9. Resume by POKE &aaaa,1 (GLOBAL from *ChainMergeMacro*)

🛈 This implementation appends CHAIN MERGE'd programs always at the end

CHAIN MERGE Example

Original code

PROGRAM-1.BAS

```
10 IF A = 1 GOTO 100
20 PRINT "program-1"
30 A = 1
40 CHAIN MERGE "!program-2"
```

PROGRAM-2.BAS

```
100 PRINT "program-2"
```

Staged code before chain merge

(4)	(3) (9)	`10 ON ERROR GOTO 50:ON BREAK GOSUB 50:REM @interrupt`
	(3) (10)	`20 SYMBOL AFTER 256:MEMORY &a66b:REM @himem`
	(3) (8)	`30 GOTO 80:REM @jump`

	(3) (7)	40 ERROR 255:REM @error
	(3) (6)	50 POKE &a679,1:POKE &eeee,ERR:END:REM @end
	(3) (13)	60 IF PEEK(&a67b)=0 GOTO 60:REM @chainmerge
	(3) (13)	70 POKE &a67b,0:RESTORE:GOTO 0:REM @chainmerge
	(3) (5)	80 REM @land
		90 IF A = 1 GOTO 140
		100 PRINT "program-1"
		110 A = 1
		120 POKE &a67a,1:GOTO 60
	(1)	130 GOTO 130:REM @bridge
	(2)	140 GOTO 0:REM @link

Preprocessed chained code

		150 PRINT "program-2"
	(1)	160 GOTO 160:REM @bridge

Staged code after chain merge

(4)	(3) (9)	10 ON ERROR GOTO 50:ON BREAK GOSUB 50:REM @interrupt
	(3) (10)	20 SYMBOL AFTER 256:MEMORY &a66b:REM @himem
	(3) (8)	30 GOTO 80:REM @jump
	(3) (7)	40 ERROR 255:REM @error
	(3) (6)	50 POKE &a679,1:POKE &eeee,ERR:END:REM @end
	(3) (13)	60 IF PEEK(&a67b)=0 GOTO 60:REM @chainmerge
	(3) (13)	70 POKE &a67b,0:RESTORE:GOTO 10:REM @chainmerge

	(3) (5)	80 REM @land
		90 IF A = 1 GOTO 140
		100 PRINT "program-1"
		110 A = 1
		120 POKE &a67a,1:GOTO 60
		130 GOTO 150:REM @bridge
		140 GOTO 150:REM @link
		150 PRINT "program-2"
	(1)	160 GOTO 160:REM @bridge